

今天 13:15

https://www.bu.edu/president/boston-university-facts-stats/  
以这个网站为例，以非计算机专业的角度来直白的讲解当前html的整体层级结构和常见的语法。

思考了 12s >

可以。你可以把 HTML 理解成一份**有层级的大纲文档**，不是一堆乱七八糟的代码。以 BU 这个页面为例，它大概像这样：

html

├ head

# 给浏览器看的信息：标题、样式、编码、SEO 等

└ body

# 用户真正能看到的网页内容

├ header

# 顶部区域：学校/办公室名称、导航菜单、搜索

├ main

# 页面主体内容：BU Facts & Stats

|

├ 页面标题

|

├ 简介文字

|

├ Quick Facts & Stats

|

├ Community

|

├ Campus

|

└ Academics

└ footer

# 底部：地址、电话、版权、隐私政策等

这个页面的顶部有“Boston University Office of the President”、导航菜单如 Meet Our President、Initiatives、Updates、Contact Us 等；主体部分是“BU Facts & Stats”，里面包含 Quick Facts & Stats、Community、Campus、Academics 等内容；底部有地址、电话、版权和隐私政策等信息。

## 1. HTML 的基本思想：一层套一层

HTML 很像 Word 文档的大纲：

<h1>BU Facts & Stats</h1>

<h4>Quick Facts & Stats</h4>

<h3>Research Expenditures</h3>

<p>\$554.0M</p>

你可以这样理解：

<h1> = 一级标题

<h4> = 四级标题 / 小标题

<h3> = 三级标题 / 某个指标名称

<p> = 普通段落文字

在 BU 页面里，BU Facts & Stats 是主标题，Quick Facts & Stats、Community、Campus、Academics 更像分组标题；具体指标包括 Research Expenditures、Study Abroad Programs、Sponsored Research Awards 等。

所以你之前问“是不是大概储存在 h4 里面”，答案是：**不完全是。**

更准确地说：

h4 可能是分类标题

h3 可能是具体指标名称

数值可能在 h3 后面的其他标签或文本节点里

li 里面可能是一条一条的列表数据

例如页面中：

https://chatgpt.com/c/6a59ba80-28a0-83e8-9f0e-0844fcbcfffd

1/22

Research Expenditures → \$554.0M

Study Abroad Programs → 200+

Sponsored Research Awards → \$645.6M

这些是“指标名称 + 数值”的结构。  Boston University

## 2. 常见标签怎么理解

<html>

整个网页的最外层。

```
<html>
...
</html>
```

可以理解为：**这是一整个网页文件。**

<head>

网页的“后台信息”，普通用户一般看不到。

```
<head>
  <title>BU Facts & Stats | Office of the President</title>
</head>
```

你写：

```
print(soup.title)
print(soup.title.get_text())
```

看到的就是 <title> 里的内容。这个通常显示在浏览器标签页上。

<body>

网页真正显示出来的主体内容。

```
<body>
  页面顶部
  页面正文
  页面底部
</body>
```

你写：

```
<> Python
```

运行

会打印出整个网页主体，所以会非常长。一般爬虫时**不要直接 print 整个 body**，而是逐步找你需要的标签。

## 3. 标题标签： h1 到 h6

HTML 里有 6 级标题：

```
<h1>最大标题</h1>
<h2>二级标题</h2>
<h3>三级标题</h3>
<h4>四级标题</h4>
<h5>五级标题</h5>
<h6>六级标题</h6>
```

在这个 BU 页面里，你会看到类似：

```
<h1>BU Facts & Stats</h1>
<h4>Quick Facts & Stats</h4>
<h3>Research Expenditures</h3>
```

直白理解：

h1: 这个页面最重要的标题  
h4: 某一块内容的分组标题  
h3: 某个具体数据项的名称



所以爬取时可以先试：

```
for tag in soup.find_all(['h1', 'h3', 'h4']):
    print(tag.name, tag.get_text(strip=True))
```

tag.name 会告诉你它是 h1 、 h3 还是 h4 。

## 4. 列表标签： ul 、 li

页面里 Community 和 Academics 部分有很多一条一条的数据，例如：

```
Student Body 37,557
Living Alumni 431,000+
Total Employees 10,674
Faculty 4,309
```



这些通常会放在列表结构里。网页中 Community 部分确实是以列表形式展示的。  Boston University

HTML 可能类似这样：

```
<ul>
  <li>Student Body 37,557</li>
  <li>Living Alumni 431,000+</li>
  <li>Total Employees 10,674</li>
</ul>
```

含义是：

ul = unordered list，无序列表  
li = list item，列表中的一项



爬取时可以这样看：

```
for item in soup.find_all('li'):
    print(item.get_text(strip=True))
```

不过注意：网页顶部导航菜单也是 li ，所以你会同时抓到导航内容和正文内容。后面如果想精准抓取，就需要限制范围。

5. 链接标签： a

网页里所有可以点击的文字，大多是 <a> 标签。

HTML

</> > ↺

含义是：

a = anchor，链接  
href = 链接地址  
标签中间的文字 = 用户看到的文字

例如页面顶部有 Meet Our President、Initiatives、Updates、Contact Us 等导航链接。  Boston University

爬虫里可以这样提取：

```
for link in soup.find_all('a'):
    text = link.get_text(strip=True)
    href = link.get('href')
    print(text, href)
```

6. 属性： class 、 id 、 href 、 src

HTML 标签不仅有名字，还可以有“附加说明”，叫属性。

例如：

```
<div class="stats-card">
  <h3>Research Expenditures</h3>
  <p>$554.0M</p>
</div>
```

这里：

div = 一个普通容器  
class="stats-card" = 这个容器的类别名

常见属性有：

| 属性    | 直白理解        | 常见用途              |
|-------|-------------|-------------------|
| class | 给一类元素起名字    | 方便 CSS 美化，也方便爬虫定位 |
| id    | 给某一个唯一元素起名字 | 类似身份证             |
| href  | 链接地址        | 用在 <a>            |
| src   | 图片或资源地址     | 用在 <img> 、 script |
| alt   | 图片说明文字      | 用在 <img>          |

例如图片标签：

HTML

</> > ↺

意思是：

src = 图片在哪里  
alt = 图片加载不出来时显示什么说明



## 7. div 和 section：网页里的“盒子”

很多网页会大量使用：

```
<div>...</div>  
<section>...</section>
```

这两个标签本身没有特别具体的含义，主要是用来**分块**。

你可以把它们理解成：

div: 普通盒子  
section: 有主题的一块内容



比如 BU 页面可能有类似这种逻辑：

```
<section>  
  <h4>Community</h4>  
  <ul>  
    <li>Student Body 37,557</li>  
    <li>Living Alumni 431,000+</li>  
  </ul>  
</section>
```

也就是说：

section 是一个内容区  
h4 是这个区的标题  
ul/li 是这个区下面的具体数据



## 8. 为什么 BeautifulSoup 输出看起来很乱？

因为网页不是为人直接阅读 HTML 代码设计的，而是给浏览器解析的。

例如你看到的页面是整洁的：

Research Expenditures  
\$554.0M



但背后的 HTML 可能是：

```
<div class="fact">  
  <div class="fact-title">  
    <h3>Research Expenditures</h3>  
  </div>  
  <div class="fact-number">  
    $ 554.0 M  
  </div>  
</div>
```

浏览器会把它排版成漂亮页面，但你用 soup.body 看到的是原始结构，所以会很长。

## 9. 对这个页面，应该怎样一步步看？

你现在不要一上来就直接抓数据，建议按这个顺序：

第一步：看网页标题

</> Python

运行

第二步：看所有标题层级

```
for tag in soup.find_all(['h1', 'h2', 'h3', 'h4']):
    print(tag.name, tag.get_text(strip=True))
```

这样你会知道页面大纲。

第三步：看所有列表项

```
for li in soup.find_all('li'):
    print(li.get_text(strip=True))
```

这一步会看到导航菜单，也会看到正文里的 Community、Academics 数据。

第四步：先人工判断数据规律

比如你可能发现：

- h3 后面常常跟着一个数值

li 里面可能直接包含 “名称 + 数值”

h4 是分类标题
- 

然后再决定怎么写爬虫。

10. 这个页面的数据结构可以这样理解

从爬虫角度看，它不是一个标准表格，而更像：

```
{
  "Quick Facts & Stats": {
    "Research Expenditures": "$554.0M",
    "Study Abroad Programs": "200+",
    "Sponsored Research Awards": "$645.6M"
  },
  "Community": {
    "Student Body": "37,557",
    "Living Alumni": "431,000+",
    "Total Employees": "10,674",
    "Faculty": "4,309"
  },
  "Campus": {
    "Classrooms": "848",
    "Buildings": "343",
    "Laboratories": "1,481",
    "Libraries": "13",
    "Campus Area (acres)": "140"
  },
  "Academics": {
    "Study Abroad Programs": "80+",
```



```
    "Average Class Size": "30",
    "Faculty": "4,309",
    "Student/Faculty Ratio": "11:1",
    "Schools and Colleges": "17",
    "Programs of Study": "300+"
  }
}
```

这就是题目说 “store the data as json file” 最终想要的方向。

## 11. 最重要的几个语法总结

你先记住这些就够了：

```
<h1>BU</h1>
```

这是最基本结构。

```
<h2>Academics</h2>
```

表示一级标题。

```
<a href="#">Link</a>
```

表示一个链接。

```
<ul>
  <li>Student Body 37,557</li>
  <li>Faculty 4,309</li>
</ul>
```

表示列表。

```
<div class="card">
  <h3>Classrooms</h3>
  <p>848</p>
</div>
```

表示一个网页小模块。

## 12. 对你现在学习爬虫最有用的理解

不要问“数据是不是都在 h4 里”，而是要问：

1. 哪些标签是分类标题？
2. 哪些标签是指标名称？
3. 哪些标签是数值？
4. 它们之间在 HTML 里是什么前后关系？

对这个 BU 页面来说，大概是：

h1: 页面主标题  
h4: 分组标题，例如 Quick Facts & Stats、Community、Campus、Academics  
h3: 部分具体指标名称，例如 Research Expenditures、Classrooms  
li: 列表型数据，例如 Student Body 37,557  
普通文本或其他标签：具体数值

所以你的下一步可以先运行：

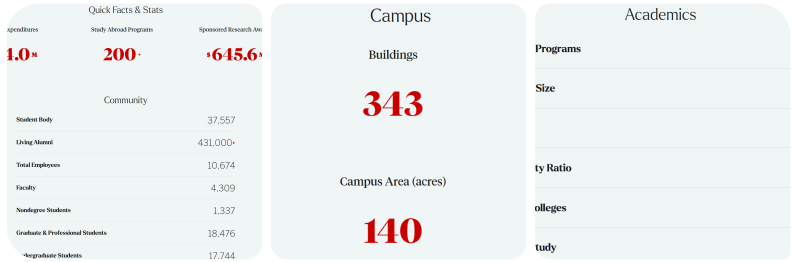
```
for tag in soup.find_all(['h1', 'h3', 'h4', 'li']):
    print(tag.name, tag.get_text(strip=True))
```

这会比 `print(soup.body)` 清楚很多。

   ...  来源

在查找所有列表项的时候，运行完出现错误  
Traceback (most recent call last):  
File "d:\Study\30-Days-of-Python-ssy\30-Days-Of-Python\22\_Day\_Web\_scraping\day\_22.py", line 50, in <module>  
for li in soup.findall('li'):  
^^^^^^^^^^^^^^^^^^^^  
TypeError: 'NoneType' object is not callable





网页如图所示。同时Quick Facts & Stats和community和campus和academics对应h4 class = 'stat-group-title'  
Research Expenditures等八个的文字对应 h3 class = 'bu-stat-title',  
它们的值对应span class = 'bu-stat-value-field'  
列表中的student body层级的文字对应 span class='stat-lable', 它们的值对应span class='stat-figure'

你的判断基本是对的。这个页面可以分成 两种数据结构 来爬：

1. 分组标题

这几个大标题：

Quick Facts & Stats

Community

Campus

Academics

对应：

HTML

所以可以这样找：

```
groups = soup.find_all('h4', class_='stat-group-title')

for group in groups:
    print(group.get_text(strip=True))
```

注意 Python 里写的是 `class_`，不是 `class`，因为 `class` 是 Python 的关键字。

## 2. 第一种数据：卡片式数据

比如：

|                           |          |
|---------------------------|----------|
| Research Expenditures     | \$554.0M |
| Study Abroad Programs     | 200+     |
| Sponsored Research Awards | \$645.6M |
| Classrooms                | 848      |
| Buildings                 | 343      |
| Laboratories              | 1,481    |
| Libraries                 | 13       |
| Campus Area (acres)       | 140      |

它们的结构是：

```
<h3 class="bu-stat-title">Research Expenditures</h3>
<span class="bu-stat-value-field">554.0</span>
```

所以可以分别找：

```
titles = soup.find_all('h3', class_='bu-stat-title')
values = soup.find_all('span', class_='bu-stat-value-field')

for title, value in zip(titles, values):
    print(title.get_text(strip=True), value.get_text(strip=True))
```

这里 `zip(titles, values)` 的意思是：  
把第一个标题配第一个数值，第二个标题配第二个数值，依次对应。

## 3. 第二种数据：列表式数据

比如 Community 和 Academics：

|                 |          |
|-----------------|----------|
| Student Body    | 37,557   |
| Living Alumni   | 431,000+ |
| Total Employees | 10,674   |
| Faculty         | 4,309    |

它们的结构是：

```
<span class="stat-lable">Student Body</span>
<span class="stat-figure">37,557</span>
```

你这里写的 `stat-lable` 注意一下，正常英文应该是 `label`，但如果网页源码里确实写的是 `stat-lable`，那代码里也必须照着网页写。

可以这样找：

```
labels = soup.find_all('span', class_='stat-lable')
figures = soup.find_all('span', class_='stat-figure')

for label, figure in zip(labels, figures):
    print(label.get_text(strip=True), figure.get_text(strip=True))
```

## 4. 先用简单代码把数据打印出来

你可以先这样写，确认能不能抓到内容：

```
import requests
from bs4 import BeautifulSoup

url = 'https://www.bu.edu/president/boston-university-facts-stats/'
response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

print('==== 分组标题 ====')
for group in soup.find_all('h4', class_='stat-group-title'):
    print(group.get_text(strip=True))

print('\n==== 卡片式数据 ====')
titles = soup.find_all('h3', class_='bu-stat-title')
values = soup.find_all('span', class_='bu-stat-value-field')

for title, value in zip(titles, values):
    print(title.get_text(strip=True), ':', value.get_text(strip=True))

print('\n==== 列表式数据 ====')
labels = soup.find_all('span', class_='stat-lable')
figures = soup.find_all('span', class_='stat-figure')

for label, figure in zip(labels, figures):
    print(label.get_text(strip=True), ':', figure.get_text(strip=True))
```

## 5. 如果要整理成 JSON，可以先这样做

```
import requests
from bs4 import BeautifulSoup
import json

url = 'https://www.bu.edu/president/boston-university-facts-stats/'
response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

data = {}

# 卡片式数据: h3 + bu-stat-value-field
card_data = {}

titles = soup.find_all('h3', class_='bu-stat-title')
values = soup.find_all('span', class_='bu-stat-value-field')

for title, value in zip(titles, values):
    key = title.get_text(strip=True)
    val = value.get_text(strip=True)
    card_data[key] = val

data['card_stats'] = card_data

# 列表式数据: stat-lable + stat-figure
list_data = {}

labels = soup.find_all('span', class_='stat-lable')
```

```
figures = soup.find_all('span', class_='stat-figure')

for label, figure in zip(labels, figures):
    key = label.get_text(strip=True)
    val = figure.get_text(strip=True)
    list_data[key] = val

data['list_stats'] = list_data

# 保存为 json 文件
with open('bu_facts_stats.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=4)

print(data)
```

生成的结构大概是：

```
{
  "card_stats": {
    "Research Expenditures": "554.0",
    "Study Abroad Programs": "200",
    "Sponsored Research Awards": "645.6",
    "Classrooms": "848",
    "Buildings": "343"
  },
  "list_stats": {
    "Student Body": "37,557",
    "Living Alumni": "431,000+",
    "Total Employees": "10,674",
    "Faculty": "4,309"
  }
}
```

## 6. 你现在需要注意的重点

这个页面不是一个简单的表格，所以不能只用一种标签抓全部数据。

更准确的理解是：

- h4.stat-group-title  
负责分组标题
- h3.bu-stat-title  
负责卡片式数据的名称
- span.bu-stat-value-field  
负责卡片式数据的数值
- span.stat-lable  
负责列表式数据的名称
- span.stat-figure  
负责列表式数据的数值



所以你现在的思路是正确的：

先观察网页源码里“文字在哪个标签里”，再用 BeautifulSoup 对应地 find\_all() 。



出现一个问题，比如Study Abroad Programs 200 + 它会把+分开来，  
然后就把+单独对应到下一条了，有一个错位

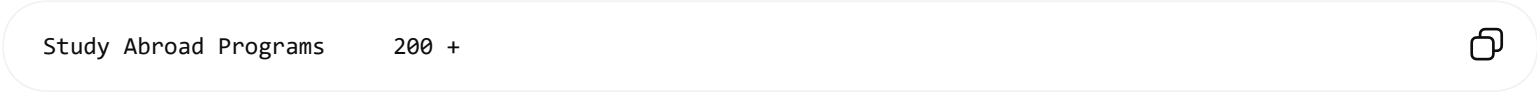
这个问题的原因是：你现在可能是这样做的：

```
titles = soup.find_all('h3', class_='bu-stat-title')
values = soup.find_all('span', class_='bu-stat-value-field')

for title, value in zip(titles, values):
    print(title.get_text(strip=True), value.get_text(strip=True))
```

这种写法的问题是：它默认一个 title 对应一个 value。

但是网页里有些值不是一个完整的文本，而是被拆成好几个小标签，比如：

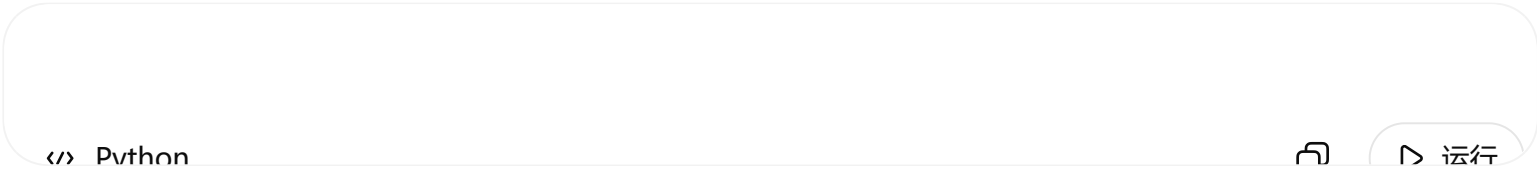


在 HTML 里可能类似：

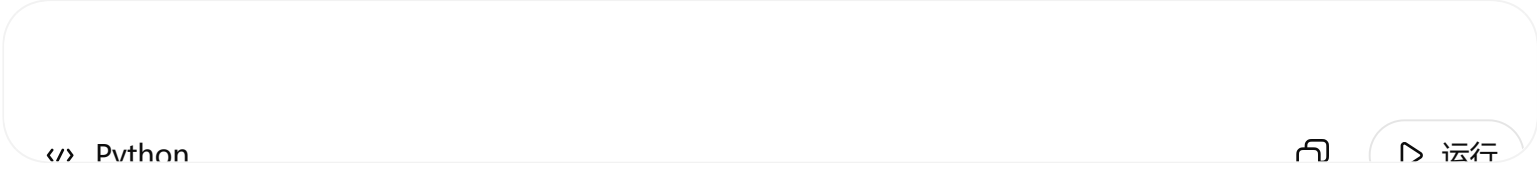
```
<h3 class="bu-stat-title">Study Abroad Programs</h3>

<span class="bu-stat-value-field">200</span>
<span class="bu-stat-value-field">+</span>
```

所以 find\_all('span', class\_='bu-stat-value-field') 找到的不是：



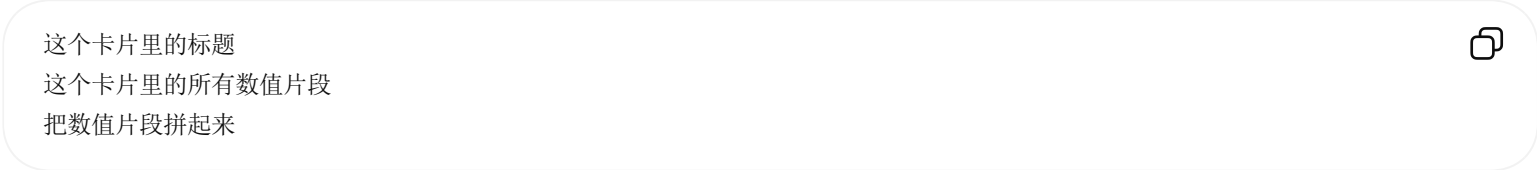
而可能是：



这样一来， zip(titles, values) 就会错位。

解决思路：不要全局配对，要在每一个小模块内部找

也就是说，不要先把所有 title 找出来、再把所有 value 找出来。  
而是应该先找到每一个统计卡片，然后在这个卡片里面找：



你可以先试这个版本：

```
card_titles = soup.find_all('h3', class_='bu-stat-title')

for title in card_titles:
    parent = title.find_parent()

    value_parts = parent.find_all('span', class_='bu-stat-value-field')

    name = title.get_text(strip=True)
    value = ''.join(part.get_text(strip=True) for part in value_parts)

    print(name, ':', value)
```

如果发现 value 为空，说明 title.find\_parent() 找到的父级太小，需要往上找一层：

```
card_titles = soup.find_all('h3', class_='bu-stat-title')

for title in card_titles:
    parent = title.find_parent().find_parent()

    value_parts = parent.find_all('span', class_='bu-stat-value-field')

    name = title.get_text(strip=True)
    value = ''.join(part.get_text(strip=True) for part in value_parts)

    print(name, ': ', value)
```

### 更稳一点的写法

如果这个页面中每一个统计卡片外面都有一个共同的 `div`，最好先找到这个 `div`。你可以先打印一下某个标题的父级看看：

```
title = soup.find('h3', class_='bu-stat-title')
print(title.find_parent())
```

或者：

```
</> Python
```

看一下外层容器是什么 `class`。

假设外层容器叫：

```
< HTML
```

那就可以这样写：

```
stats = soup.find_all('div', class_='bu-stat')

for stat in stats:
    title = stat.find('h3', class_='bu-stat-title')
    value_parts = stat.find_all('span', class_='bu-stat-value-field')

    if title and value_parts:
        name = title.get_text(strip=True)
        value = ''.join(part.get_text(strip=True) for part in value_parts)

        print(name, ': ', value)
```

这样 `200` 和 `+` 就会被拼成：

```
Study Abroad Programs : 200+
```

不会把 `+` 错配给下一条。

### 列表式数据也建议用同样思路

对于 `Community` 和 `Academics` 这种：

```
Student Body      37,557
Living Alumni     431,000+
```

你之前用：



```
labels = soup.find_all('span', class_='stat-lable')
figures = soup.find_all('span', class_='stat-figure')
```

一般可以，但如果以后也遇到错位，最好改成“按每一行来找”。

比如如果每一行外面是一个 li：

```
items = soup.find_all('li')

for item in items:
    label = item.find('span', class_='stat-lable')
    figure = item.find('span', class_='stat-figure')

    if label and figure:
        name = label.get_text(strip=True)
        value = figure.get_text(strip=True)
        print(name, ': ', value)
```

这个逻辑更安全，因为它是：

在同一个 li 里面找名称和值



而不是：

先找全网页所有名称，再找全网页所有值，然后强行配对



## 核心区别

不太稳的写法：

```
titles = soup.find_all(...)
values = soup.find_all(...)

for title, value in zip(titles, values):
    ...
```

这种适合非常整齐的网页。

更稳的写法：

```
for container in containers:
    title = container.find(...)
    values = container.find_all(...)
```

这种适合真实网页，因为真实网页经常把 \$、数字、M、+ 拆在不同标签里。

... 来源

层级为Quick Facts & Stats h4 class = 'stat-group-title'  
接下来Research Expenditures文字对应 h3 class = 'bu-stat-title'，值  
对应span class = 'bu-stat-value-field'  
和上面同一层级但是不用样式的为  
student body文字对应 span class='stat-lable'，值对应span  
class='stat-figure'

思考了 30s >

对，你现在描述的结构可以理解为：

```
h4.stat-group-title
  一组数据的大标题

  h3.bu-stat-title
  span.bu-stat-value-field

  或者：

  span.stat-lable
  span.stat-figure
```

也就是说， `h4.stat-group-title` 是“分组名”，下面的数据有两种形式。

关键问题是：**不要把全网页所有 `title` 和 `value` 分别找出来再 `zip` 配对。**  
因为 `200+` 这种值可能被拆成：

```
200
+
```

所以全局 `zip()` 很容易错位。

### 更稳的思路

应该按照这个逻辑爬：

```
先找到一个 h4 分组标题
然后只在这个分组下面找数据
遇到下一个 h4 时，说明当前分组结束
```

然后在每个分组里分别处理两种数据：

```
第一种: h3.bu-stat-title + span.bu-stat-value-field
第二种: span.stat-lable + span.stat-figure
```

### 可以这样写

```
import requests
from bs4 import BeautifulSoup
import json

url = 'https://www.bu.edu/president/boston-university-facts-stats/'
response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

data = {}

group_titles = soup.find_all('h4', class_='stat-group-title')

for group in group_titles:
    group_name = group.get_text(strip=True)
    data[group_name] = {}

    # 从当前 h4 后面开始找
    for tag in group.find_all_next():

        # 如果遇到下一个 h4.stat-group-title，说明当前 group 结束
        if tag.name == 'h4' and 'stat-group-title' in tag.get('class', []) and tag != group:
            break

    # 第一种结构: h3.bu-stat-title + span.bu-stat-value-field
```



```
if tag.name == 'h3' and 'bu-stat-title' in tag.get('class', []):
    title = tag.get_text(strip=True)

    value_parts = []

    # 从这个 h3 后面继续找，直到遇到下一个 h3 或 h4
    for next_tag in tag.find_all_next():

        if next_tag.name == 'h3' and 'bu-stat-title' in next_tag.get('class', []):
            break

        if next_tag.name == 'h4' and 'stat-group-title' in next_tag.get('class', []):
            break

        if next_tag.name == 'span' and 'bu-stat-value-field' in next_tag.get('class', []):
            value_parts.append(next_tag.get_text(strip=True))

    value = ''.join(value_parts)
    data[group_name][title] = value

# 第二种结构: span.stat-lable + span.stat-figure
if tag.name == 'span' and 'stat-lable' in tag.get('class', []):
    label = tag.get_text(strip=True)

    parent = tag.find_parent()
    figure_tag = parent.find('span', class_='stat-figure')

    if figure_tag:
        figure = figure_tag.get_text(strip=True)
        data[group_name][label] = figure

print(data)

with open('bu_facts_stats.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=4)
```

这段代码为什么能解决 200+ 错位？

重点在这里：

</> Python

运行

它不是只找一个 value，而是把当前标题下面的所有 span.bu-stat-value-field 都找出来。

比如网页里如果是：

```
<span class="bu-stat-value-field">200</span>
<span class="bu-stat-value-field">+</span>
```

代码会得到：

</> Python

运行

然后：

</> Python

运行

就会变成：

200+

运行

这样 + 就不会被错误地分给下一条数据。

### 你要特别注意这个拼写

你写的是：

</> Pvthon

运行

正常英文应该是：

</> Python

运行

但是爬虫必须按照网页源码里的 class 名写。  
所以如果网页源码里确实是 stat-lable , 代码就必须写：

</> Python

运行

不能自己改成 stat-label 。

### 最终 JSON 大概会变成这样

```
{
  "Quick Facts & Stats": {
    "Research Expenditures": "554.0",
    "Study Abroad Programs": "200+",
    "Sponsored Research Awards": "645.6"
  },
  "Community": {
    "Student Body": "37,557",
    "Living Alumni": "431,000+",
    "Total Employees": "10,674",
    "Faculty": "4,309"
  },
  "Campus": {
    "Classrooms": "848",
    "Buildings": "343",
    "Laboratories": "1,481",
    "Libraries": "13",
    "Campus Area (acres)": "140"
  },
  "Academics": {
    "Study Abroad Programs": "80+",
    "Average Class Size": "30",
    "Faculty": "4,309",
    "Student/Faculty Ratio": "11:1",
    "Schools and Colleges": "17",
    "Programs of Study": "300+"
  }
}
```

这个页面主体确实是这几个分组：Quick Facts & Stats、Community、Campus、Academics；其中 Quick Facts & Stats 和 Campus 是卡片式数据，Community 和 Academics 是列表式数据。  bu.edu

   ...  来源

输出结果是这样  
{ 'Quick Facts & Stats': { 'Research Expenditures': '\$554.0M', 'Study

Abroad Programs': '200+', 'Sponsored Research Awards': '\$645.6M'}, 'Community': {}, 'Campus': {'Classrooms': '848', 'Buildings': '343', 'Laboratories': '1,481', 'Libraries': '13', 'Campus Area (acres)': '140'}, 'Academics': {}}

有两个类的没有输出

这个结果说明：**卡片式数据已经成功抓到了**，但是列表式数据这一部分没有被识别到。

你现在输出是：

```
'Community': {}
'Academics': {}
```

说明代码没有找到这一类标签：

```
</> Python   运行
```

最可能的问题是：**class 名写错了**。

你写的是：

```
</> Python   运行
```

但网页里很可能实际是：

```
</> Python   运行
```

label 的正确拼写是 label，不是 lable。

### 先检查真实 class 名

你可以运行这一段，专门看 Student Body 到底在哪个标签里：

```
for span in soup.find_all('span'):
    text = span.get_text(strip=True)
    if 'Student Body' in text:
        print(span)
        print(span.get('class'))
```

如果输出类似：

```
</> Python   运行
```

那就说明应该写：

```
</> Python   运行
```

而不是：

```
</> Python
```

</> Python



运行

## 把这一段改掉

原来可能是：

</> Python



运行

改成：

</> Python



运行

以及：

</> Python



运行

## 推荐你用这个更稳的版本

```
import requests
from bs4 import BeautifulSoup
import json

url = 'https://www.bu.edu/president/boston-university-facts-stats/'
response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

data = {}

group_titles = soup.find_all('h4', class_='stat-group-title')

for group in group_titles:
    group_name = group.get_text(strip=True)
    data[group_name] = {}

    for tag in group.find_all_next():

        # 遇到下一个 h4, 说明当前分组结束
        if tag.name == 'h4' and 'stat-group-title' in tag.get('class', []) and tag != group:
            break

        # 第一种: h3.bu-stat-title + span.bu-stat-value-field
        if tag.name == 'h3' and 'bu-stat-title' in tag.get('class', []):
            title = tag.get_text(strip=True)

            value_parts = []

            for next_tag in tag.find_all_next():

                if next_tag.name == 'h3' and 'bu-stat-title' in next_tag.get('class', []):
                    break

                if next_tag.name == 'h4' and 'stat-group-title' in next_tag.get('class', []):
                    break

                if next_tag.name == 'span' and 'bu-stat-value-field' in next_tag.get('class', []):
                    value_parts.append(next_tag.get_text(strip=True))

            value = ''.join(value_parts)
            data[group_name][title] = value
```

```
# 第二种: span.stat-label + span.stat-figure
if tag.name == 'span' and 'stat-label' in tag.get('class', []):
    label = tag.get_text(strip=True)

    parent = tag.find_parent()
    figure_tag = parent.find('span', class_='stat-figure')

    if figure_tag:
        figure = figure_tag.get_text(strip=True)
        data[group_name][label] = figure

print(data)

with open('bu_facts_stats.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=4)
```

## 如果还是没有输出 Community 和 Academics

再运行这个检查：

```
for span in soup.find_all('span'):
    text = span.get_text(strip=True)
    if text in ['Student Body', 'Living Alumni', 'Average Class Size', 'Programs of Study']:
        print(text)
        print(span)
        print(span.get('class'))
```

它会直接告诉你这些文字对应的真实 class。

## 你这个结果说明了什么？

现在：

```
'Quick Facts & Stats': {...}
'Campus': {...}
```

能出来，说明这部分没有问题：

```
h3.bu-stat-title
span.bu-stat-value-field
```

但是：

```
'Community': {}
'Academics': {}
```

为空，说明问题集中在这一部分：

```
span.stat-label
span.stat-figure
```

所以优先检查 stat-label 这个 class 名。通常就是拼写问题。

📄 ⬆️ ↺️ ⋮

确实是拼写问题，其他没错

对，那就说明你的整体思路是正确的，问题只是在 class 名必须和网页源码**完全一致**。

把所有这里：

</> Python

运行

改成：

</> Python

运行

即可。

比如列表式数据部分应该是：

</> Python

```
if tag.name == 'span' and 'stat-label' in tag.get('class', []):
    label = tag.get_text(strip=True)

    parent = tag.find_parent()
    figure_tag = parent.find('span', class_='stat-figure')

    if figure_tag:
        figure = figure_tag.get_text(strip=True)
        data[group_name][label] = figure
```

运行

或者更简单地理解：

span.stat-label → Student Body / Faculty / Programs of Study

span.stat-figure → 37,557 / 4,309 / 300+

这个错误很常见，爬虫里 class 名、标签名、属性名都必须照着网页源码写，哪怕只差一个字母，BeautifulSoup 也会找不到。

你现在的结果已经证明：

Quick Facts & Stats 成功

Campus 成功

Community / Academics 原来为空

不是层级逻辑错了，而是列表部分的 class 名没有匹配上。改完后，Community 和 Academics 应该就能正常输出了。

📄 ⬆️ ↺️ ⋮